Practical 1

```
#include <iostream>
using namespace std;
// Global variable to count recursive calls
int callCount = 0;
// Recursive Fibonacci function
int fibonacci(int n) {
    callCount++;  // Increment count for each call
    if (n == 0)
        return 0;
     if (n == 1)
        return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
int main() {
    int limit;
    cout << "Enter the number of terms: ";
    cin >> limit;
    cout << "Fibonacci Series: ";
    for (int i = 0; i < limit; i++) {
        cout << fibonacci(i) << " ";
    }
    cout << endl;
    cout << "Total Recursive Calls: " << callCount << endl;
    return 0;
}
```

Output :-

```
Enter the number of terms: 8
Fibonacci Series: 0 1 1 2 3 5 8 13
Total Recursive Calls: 100


=== Code Execution Successful ===
```

Practical 1

```
#include <stdio.h>
#include <time.h>
// Iterative function for factorial
long long factorial_iterative(int n) {
long long result = 1;
int i;
for (i = 1; i <= n; i++) \
result *= i;
return result;
}
// Recursive function for factorial
long long factorial_recursive(int n) {
if (n == 0 || n == 1)
return 1;
else
return n * factorial_recursive(n - 1);
}
int main() {
int n;
clock_t start, end;
double cpu_time_used;
long long result;
printf("Harshwardhan D:\n");
printf("PRN 32558013205:\n");
printf("Enter a number to find factorial: ");
scanf("%d", &n);
// Iterative function timing
start = clock();
result = factorial_iterative(n);
end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
```

```
printf("\n[Iterative] Factorial of %d = %lld", n, result);
printf("\n[Iterative] Time taken: %f seconds", cpu_time_used);
printf("\n[Iterative] Space Complexity˜ ˜ O(1)\n");
// Recursive function timing
start = clock();
result = factorial_recursive(n);
end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\n[Recursive] Factorial of %d = %lld", n, result);
printf("\n[Recursive] Time taken: %f seconds", cpu_time_used);
printf("\n[Recursive] Space Complexity˜ ˜ O(n) (due to recursion stack)\n");
return 0;
}
```

Output:-

```
Harshwardhan D:
PRN 32558013205:
Enter a number to find factorial: 10

[Iterative] Factorial of 10 = 3628800
[Iterative] Time taken: 0.000002 seconds
[Iterative] Space Complexity = O(1)

[Recursive] Factorial of 10 = 3628800
[Recursive] Time taken: 0.000001 seconds
[Recursive] Space Complexity = O(n) (due to recursion stack)


=== Code Execution Successful ===
```

Practical 2

```
#include <stdio.h>
#include <time.h>
// Recursive Binary Search Function
int binarySearch(int arr[], int low, int high, int key) {
    if (low <= high) {
        int mid = low + (high - low) / 2;
if (arr[mid] == key)
            return mid;
if (arr[mid] > key)
            return binarySearch(arr, low, mid - 1, key
    return binarySearch(arr, mid + 1, high, key);
    }
    return -1;
}
int main() {
    int arr[100000], n, key, result;
    clock_t start, end;
    double cpu_time_used;
    int i;
  printf("Recursive Binary Search\n");
    printf("----------------------\n");
// Your details
    printf("\nName : Harshwardhan D \n");
    printf("PRN : 32558013205\n\n");
   printf("Enter number of elements: ");
    scanf("%d", &n);
   printf("Enter %d elements in sorted order:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
  printf("Enter the element to search: ");
    scanf("%d", &key);

    start = clock();
    result = binarySearch(arr, 0, n - 1, key);
    end = clock();
cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
if (result != -1)
        printf("\nElement %d found at position %d.\n", key, result + 1);
    else
        printf("\nElement %d not found in the array.\n", key);
printf("\nTime taken by recursive binary search: %.10f seconds", cpu_time_used);
```

```
    printf("\nTheoretical Time Complexity: O(log n)");
    printf("\nSpace Complexity (due to recursion stack): O(log n)\n");
return 0;
}
```

Output:-

```
Recursive Binary Search
-----------------------

Name :Harshwardhan D
PRN : 32558013205

Enter number of elements: 4
Enter 4 elements in sorted order:
10
11
12
13
Enter the element to search: 10

Element 10 found at position 1.

Time taken by recursive binary search: 0.0000030000 seconds
Theoretical Time Complexity: O(log n)
Space Complexity (due to recursion stack): O(log n)


=== Code Execution Successful ===
```

Practical 3

```
#include <stdio.h>
#include <time.h>
#define MAX 100 // maximum size
void bubbleSort(int a[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
     for (j = 0; j < n - i - 1; j++) {
       if (a[j] > a[j + 1]) {
          temp = a[j];
          a[j] = a[j + 1];
          a[j + 1] = temp;
       }
     }
   }
}
void insertionSort(int a[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
      key = a[i];
      j = i - 1;
      while (j >= 0 && a[j] > key) {
         a[j + 1] = a[j];
         j--;
      }
      a[j + 1] = key;
   }}
// Function to display array
void displayArray(int a[],int n) {
int i;
 for (i = 0; i < n; i++)
```

```
printf("%d ", a[i]);
 }
 printf("\n");
}
int main(){
int n,i;
int a[MAX],b[MAX];
clock_t start, end;
double bubbleTime,insertionTime;
printf("SortingAlgorithm Comparison\n");
printf("-----------------------\n");
//??Yourdetailsadded here
printf("\nName: Harshwardhan D\n\n");
printf("PRN: 32558013205\n\n");
printf("Enter numberof elements: ");
scanf("%d", &n);
if(n >MAX){
   printf("Too many elements! Max = %d\n", MAX);
return 0;
}
printf("Enterelements:\n");
for (i = 0; i < n; i++) {
   scanf("%d", &a[i]);
     b[i] = a[i];
}
//Bubble Sort Time
start = clock();
bubbleSort(a, n);
end = clock();
bubbleTime = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nSorted Array using Bubble Sort:\n");
displayArray(a, n);
//Insertion Sort Time start =
clock();
insertionSort(b, n);
```

```
= clock();
insertionTime = (double)(end - start) / CLOCKS_PER_SEC;
printf("\nSorted Array using Insertion Sort:\n");
displayArray(b, n);
//Display time taken
printf("\nBubble Sort Time: %f seconds", bubbleTime);
printf("\nInsertion Sort Time: %f seconds\n", insertionTime);
return 0;
}
```

Output:-

```
Sorting Algorithm Comparison
-------------------------------------

Name:Harshwardhan D
PRN: 32558013205

Enter number of elements: 5
Enter elements:
2
3
4
5
1

Sorted Array using Bubble Sort:
1 2 3 4 5

Sorted Array using Insertion Sort:
1 2 3 4 5

Bubble Sort Time: 0.000002 seconds
Insertion Sort Time: 0.000002 seconds


=== Code Execution Successful ===
```

Practical 4

```
#include <stdio.h>
#include <stdlib.h>
struct MinHeapNode {
   char data;
   unsigned freq;
   struct MinHeapNode *left, *right;
};
struct MinHeap {
   unsigned size;
   unsigned capacity;
   struct MinHeapNode** array;
};
struct MinHeapNode* newNode(char data, unsigned freq) {
   struct MinHeapNode* temp = (struct MinHeapNode*)malloc(sizeof(struct MinHeapNode));
   temp->left = temp->right = NULL;
   temp->data = data;
   temp->freq = freq;
   return temp;
}
struct MinHeap* createMinHeap(unsigned capacity) {
   struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
   minHeap->size = 0;
   minHeap->capacity = capacity;
   minHeap->array = (struct MinHeapNode**)malloc(minHeap->capacity * sizeof(struct MinHeapNode*));
   return minHeap;
}
void swapMinHeapNode(struct MinHeapNode** a, struct MinHeapNode** b) {
   struct MinHeapNode* t = *a;
   *a = *b;
   *b = t;
}
void minHeapify(struct MinHeap* minHeap, int idx) {
   int smallest = idx;
   int left = 2 * idx + 1;
   int right = 2 * idx + 2;
   if (left < minHeap->size && minHeap->array[left]->freq < minHeap->array[smallest]->freq)
      smallest = left;
   if (right < minHeap->size && minHeap->array[right]->freq < minHeap->array[smallest]->freq)
      smallest = right;
   if (smallest != idx) {
      swapMinHeapNode(&minHeap->array[smallest], &minHeap->array[idx]);
      minHeapify(minHeap, smallest);
   }
}
```

```
struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
   struct MinHeapNode* temp = minHeap->array[0];
   minHeap->array[0] = minHeap->array[minHeap->size - 1];
   --minHeap->size;
   minHeapify(minHeap, 0);
   return temp;
}
void insertMinHeap(struct MinHeap* minHeap, struct MinHeapNode* minHeapNode) {
   ++minHeap->size;
   int i = minHeap->size - 1;
   while (i && minHeapNode->freq < minHeap->array[(i - 1) / 2]->freq) {
      minHeap->array[i] = minHeap->array[(i - 1) / 2];
      i = (i - 1) / 2;
   }
   minHeap->array[i] = minHeapNode;
}
struct MinHeapNode* buildHuffmanTree(char data[], int freq[], int size) {
   struct MinHeapNode *left, *right, *top;
   struct MinHeap* minHeap = createMinHeap(size);
   for (int i = 0; i < size; ++i)
      minHeap->array[i] = newNode(data[i], freq[i]);
   minHeap->size = size;
   while (minHeap->size != 1) {
      left = extractMin(minHeap);
      right = extractMin(minHeap);
      top = newNode('$', left->freq + right->freq);
      top->left = left;
      top->right = right;
      insertMinHeap(minHeap, top);
   }
   return extractMin(minHeap);
}
void printCodes(struct MinHeapNode* root, int arr[], int top) {
   if (root->left) {
      arr[top] = 0;
      printCodes(root->left, arr, top + 1);
   }
   if (root->right) {
      arr[top] = 1;
      printCodes(root->right, arr, top + 1);
   }

   if (!(root->left) && !(root->right)) {
      printf("%c: ", root->data);
      for (int i = 0; i < top; ++i)
         printf("%d", arr[i]);
      printf("\n");
```

```
    }
}

int main() {
    char arr[] = {'A', 'B', 'C', 'D', 'E', 'F'};
    int freq[] = {5, 9, 12, 13, 16, 45};
    int size = sizeof(arr) / sizeof(arr[0]);
    struct MinHeapNode* root = buildHuffmanTree(arr, freq, size);
    int code[100], top = 0;
    printf("Huffman Codes are:\n");
    printCodes(root, code, top);

  return 0;
}
```

Output:-

```
Huffman Coding Algorithm
----------------------------------------

Name : Harshwardhan D
PRN : 32558013205

Huffman Codes:
a: 0
b: 100
d: 1010
f: 10110
e: 10111
c: 11


=== Code Execution Successful ===
```

Practical 5

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define INF 9999
#define MAX 100
// ---------- STRUCTURE FOR EDGE (Used in Kruskal) ----------
typedef struct {
    int u, v, w;
} Edge;
// ---------- UNION-FIND FUNCTIONS (for Kruskal) ----------
int find(int parent[], int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}
void unionSets(int parent[], int u, int v) {
    int uRoot = find(parent, u);
    int vRoot = find(parent, v);
    parent[uRoot] = vRoot;
}
// ---------- PRIM'S ALGORITHM ----------
void primMST(int cost[MAX][MAX], int n) {
    int selected[MAX] = {0};
    int no_edge = 0, x, y;
    int totalCost = 0;
    selected[0] = 1; // Start from vertex 0
    printf("\nPrim’s Algorithm Edges:\n");
    while (no_edge < n - 1) {
        int min = INF;
        x = 0;
        y = 0;
        for (int i = 0; i < n; i++) {
            if (selected[i]) {
            for (int j = 0; j < n; j++) {
                if (!selected[j] && cost[i][j]) {
                    if (min > cost[i][j]) {
```

```
                    min = cost[i][j];
                    x = i;
                    y = j;
                }
            }
        }
    printf("Edge %d: (%d - %d) cost: %d\n", no_edge + 1, x, y, cost[x][y]);
        totalCost += cost[x][y];
        selected[y] = 1;
        no_edge++;
    }
    printf("Total cost using Prim's Algorithm = %d\n", totalCost);
}
// ---------- KRUSKAL'S ALGORITHM ----------
void kruskalMST(int cost[MAX][MAX], int n) {
    Edge edges[MAX * MAX];
    int edgeCount = 0;
    // Convert adjacency matrix to edge list
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (cost[i][j] != 0 && cost[i][j] != INF) {
                edges[edgeCount].u = i;
                edges[edgeCount].v = j;
                edges[edgeCount].w = cost[i][j];
                edgeCount++;
            }
    }
  // Sort edges by weight (simple bubble sort)
    for (int i = 0; i < edgeCount - 1; i++) {
        for (int j = 0; j < edgeCount - i - 1; j++) {
            if (edges[j].w > edges[j + 1].w) {
                Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
    }
    int parent[MAX];
    for (int i = 0; i < n; i++)
```

```
        parent[i] = i;
    int totalCost = 0, edgeAdded = 0;
    printf("\nKruskal's Algorithm Edges:\n");
    for (int i = 0; i < edgeCount && edgeAdded < n - 1; i++) {
        int uRoot = find(parent, edges[i].u);
        int vRoot = find(parent, edges[i].v);
        if (uRoot != vRoot) {
            printf("Edge %d: (%d - %d) cost: %d\n", edgeAdded + 1, edges[i].u, edges[i].v, edges[i].w);
            totalCost += edges[i].w;
            unionSets(parent, uRoot, vRoot);
            edgeAdded++;
        }
    printf("Total cost using Kruskal's Algorithm = %d\n", totalCost);
}
// ---------- MAIN FUNCTION ----------
int main() {
    int n, cost[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &n);
printf("Enter the cost adjacency matrix (use 0 for no edge):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j)
                cost[i][j] = INF;
        }
    // Measure execution time for Prim's
    clock_t start, end;
    double primTime, kruskalTime;
    start = clock();
    primMST(cost, n);
    end = clock();
    primTime = ((double)(end - start)) / CLOCKS_PER_SEC;
    // Measure execution time for Kruskal's
    start = clock();
    kruskalMST(cost, n);
    end = clock();
```

```
    kruskalTime = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\n⏱ Execution Time Comparison:");
    printf("\nPrim's Algorithm Time: %.10f seconds", primTime);
    printf("\nKruskal's Algorithm Time: %.10f seconds\n",   kruskalTime);
  return 0;
}
```

Output:-

```
Minimum Spanning Tree (Prim's & Kruskal's)
------------------------------------------

Name : Harshwardhan D
PRN  : 32558013205

Enter number of vertices: 2
Enter cost adjacency matrix (0 for no edge):
6
8
8
7

Prim's Algorithm Edges:
Edge 1 : (0 - 1) cost : 8
Total cost using Prim's Algorithm = 8

Kruskal's Algorithm Edges:
Edge 1 : (0 - 1) cost : 8
Total cost using Kruskal's Algorithm = 8

Execution Time Comparison:
Prim's Algorithm Time   : 0.0000430000 seconds
Kruskal's Algorithm Time: 0.0000610000 seconds
```

Practical 6

```
#include <stdio.h>
// Function to return maximum of two numbers
int max(int a, int b) {
    return (a > b) ? a : b;
}
// Function to solve 0/1 Knapsack using Dynamic Programming
int knapSack(int W, int wt[], int val[], int n) {
    int i, w;
    int K[n + 1][W + 1];
    // Build table K[][] in bottom-up manner
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }
    // Display DP Table
    printf("\nDynamic Programming Table:\n");
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            printf("%3d ", K[i][w]);
        }
        printf("\n");
    }
    return K[n][W];  // Final maximum value
}
int main() {
    int val[20], wt[20], W, n, i;
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter values (profits) of %d items:\n", n);
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &val[i]);
    }
    printf("Enter weights of %d items:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }
    printf("Enter capacity of Knapsack: ");
    scanf("%d", &W);
    int maxValue = knapSack(W, wt, val, n);
    printf("\nMaximum value that can be put in the knapsack = %d\n", maxValue);
    return 0;
}
```

Output:-

```
0/1 Knapsack Problem using Dynamic Programming
----------------------------------------------
Name :Harshwardhan D

PRN : 32558013205

Enter number of items: 3
Enter values (profits) of 3 items:
1 2 5
Enter weights of 3 items:
1 9 8
Enter capacity of Knapsack: 3

Dynamic Programming Table:
  0   0   0   0
  0   1   1   1
  0   1   1   1
  0   1   1   1

Maximum value that can be put in the knapsack = 1


=== Code Execution Successful ===
```

Practical 7

```
#include <stdio.h>
#include <string.h>
int max(int a, int b) { return (a > b)? a : b; }
void LCS(char X[], char Y[], int m, int n) {
   int dp[m+1][n+1];
   for (int i = 0; i <= m; i++) {
      for (int j = 0; j <= n; j++) {
         if (i == 0 || j == 0)
            dp[i][j] = 0;
         else if (X[i-1] == Y[j-1])
            dp[i][j] = dp[i-1][j-1] + 1;
         else
            dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
      }
   }
   printf("Length of LCS is %d\n", dp[m][n]);
}
int main() {
   char X[] = "AGGTAB";
   char Y[] = "GXTXAYB";
   int m = strlen(X);
   int n = strlen(Y);
   LCS(X, Y, m, n);
   return 0;
}
```

Output:-

```
Longest Common Subsequence (LCS)
--------------------------------

Name  : Harshwardhan D
PRN   : 32558013205

String 1: AGGTAB
String 2: GXTXAYB

Length of LCS is 4


=== Code Execution Successful ===
```

Practical 8

```
#include <stdio.h>
#define MAX 10

int adj[MAX][MAX], visited[MAX], n;

// BFS Function
void bfs(int start) {
   int queue[MAX], front = 0, rear = 0;
   visited[start] = 1;
   queue[rear++] = start;
   printf("BFS Traversal: ");

   while (front < rear) {
      int node = queue[front++];
      printf("%d ", node);

      for (int i = 0; i < n; i++) {
         if (adj[node][i] && !visited[i]) {
            queue[rear++] = i;
            visited[i] = 1;
         }
      }
   }
   printf("\n");
}

// DFS Function
void dfs(int start) {
   printf("%d ", start);
   visited[start] = 1;

   for (int i = 0; i < n; i++) {
      if (adj[start][i] && !visited[i]) {
         dfs(i);
      }
   }
}

int main() {
   int i, j, start;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
```

```
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    printf("Enter starting vertex: ");
    scanf("%d", &start);

    for (i = 0; i < n; i++) visited[i] = 0;
    bfs(start);

    for (i = 0; i < n; i++) visited[i] = 0;
    printf("DFS Traversal: ");
    dfs(start);
    printf("\n");

    return 0;
}
```

Output:-

```
Graph Traversal using BFS and DFS
-----------------------------------

Name  : Harashwardhan D
PRN   : 32558013205

Enter number of vertices: 4
Enter adjacency matrix:
1 2 3 4
1 4 7 2
5 8 9 6
3 2 1 4
Enter starting vertex: 4
BFS Traversal: 4
DFS Traversal: 4


=== Code Execution Successful ===
```

Practical 9

```
#include <stdio.h>

#define V 4      // Number of vertices
#define INF 99999 // Infinity (large value)

// Function to print the distance matrix
void printSolution(int dist[V][V]) {
   printf("\nShortest distances between every pair of vertices:\n");
   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         if (dist[i][j] == INF)
            printf("%7s", "INF");
         else
            printf("%7d", dist[i][j]);
      }
      printf("\n");
   }
}
// Function to implement Floyd–Warshall Algorithm
void floydWarshall(int graph[V][V]) {
   int dist[V][V]; // Matrix to store shortest path distances
   // Step 1: Initialize distance matrix same as input graph
   for (int i = 0; i < V; i++)
      for (int j = 0; j < V; j++)
         dist[i][j] = graph[i][j];
   // Step 2: Update distance matrix using dynamic programming
   for (int k = 0; k < V; k++) {       // Pick all vertices as intermediate
      for (int i = 0; i < V; i++) {    // Pick all vertices as source
         for (int j = 0; j < V; j++) { // Pick all vertices as destination
            if (dist[i][k] + dist[k][j] < dist[i][j])
               dist[i][j] = dist[i][k] + dist[k][j];
         }
      }
   }
   // Step 3: Print the final shortest distance matrix
```

```
    printSolution(dist);
}
int main() {
    /* Input weighted graph as adjacency matrix
       0 indicates self-loop
       INF means no direct edge between the vertices */
    int graph[V][V] = {
        {0,   5,  INF, 10},
       {INF, 0,   3,  INF},
       {INF, INF, 0,   1},
       {INF, INF, INF, 0}
    };
    printf("Floyd–Warshall Algorithm Implementation\n");
    printf("--------------------------------------\n");
   floydWarshall(graph);

    return 0;
}
```

Output:-

```
Floyd-Warshall Algorithm Implementation
---------------------------------------

Name :Harshwardhan D
PRN  : 32558013205

Shortest distances between every pair of vertices:
      0      5      8      9
    INF      0      3      4
    INF    INF      0      1
    INF    INF    INF      0


=== Code Execution Successful ===
```

Practical 10

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
#include <math.h>
#define N 4  // Number of cities
#define INF 9999
int costMatrix[N][N] = {
    {0, 10, 15, 20},
    {10, 0, 35, 25},
    {15, 35, 0, 30},
    {20, 25, 30, 0}
};
// ---------- FUNCTION: Find Minimum Cost Using Brute Force (Exact TSP) ----------
int minCost = INF;
int visitedAll = (1 << N) - 1;
int dp[1 << N][N];  // DP for memoization
int tspExact(int mask, int pos) {
    if (mask == visitedAll)
        return costMatrix[pos][0];  // return to starting point
    if (dp[mask][pos] != -1)
        return dp[mask][pos];
    int ans = INF;
    for (int city = 0; city < N; city++) {
        if ((mask & (1 << city)) == 0) {  // if not visited
            int newAns = costMatrix[pos][city] + tspExact(mask | (1 << city), city);
            if (newAns < ans)
                ans = newAns;
}
    dp[mask][pos] = ans;
    return ans;
}
// ---------- FUNCTION: Find Approximate Solution using Nearest Neighbor ----------
int tspApprox() {
    bool visited[N] = {false};
    int totalCost = 0;
    int current = 0;  // Start from city 0
```

```
    visited[0] = true;
    printf("\nNearest Neighbor Path: 0");
    for (int count = 1; count < N; count++) {
        int nearest = -1;
        int minDist = INF;
        for (int j = 0; j < N; j++) {
            if (!visited[j] && costMatrix[current][j] < minDist && costMatrix[current][j] != 0) {
                minDist = costMatrix[current][j];
                nearest = j;
            }
        visited[nearest] = true;
        totalCost += minDist;
        current = nearest;
        printf(" -> %d", current);
    }
    totalCost += costMatrix[current][0]; // return to start
    printf(" -> 0\n");
    return totalCost;
}
// ---------- MAIN FUNCTION ----------
int main() {
    int i, j;
    for (i = 0; i < (1 << N); i++)
        for (j = 0; j < N; j++)
            dp[i][j] = -1;
    printf("Traveling Salesperson Problem (TSP) Comparison\n");
    printf("-------------------------------------------\n");
    // Exact solution using Dynamic Programming (Bitmask)
    int exactCost = tspExact(1, 0);
    // Approximation using Nearest Neighbor
    int approxCost = tspApprox();
    // Compute approximation error
    double errorPercent = ((double)(approxCost - exactCost) / exactCost) * 100.0;
    printf("\nOptimal (Exact) TSP Cost = %d", exactCost);
    printf("\nApproximate (Nearest Neighbor) TSP Cost = %d", approxCost);
    printf("\nApproximation Error = %.2f%%\n", errorPercent);

    return 0;
```

}

Output:-

```
Traveling Salesperson Problem (TSP) Comparison
----------------------------------------------

Name : Harshwardhan D
PRN  : 32558013205

Nearest Neighbor Path: 0 -> 3 -> 2 -> 1 -> 0

Optimal (Exact) TSP Cost = 23
Approximate (Nearest Neighbor) TSP Cost = 23
Approximation Error = 0.00%


=== Code Execution Successful ===
```